AuNW Analysis

Design, mathematics and use of a deep-learning system for measuring the length and width of individual gold nanowires in SEM images

Technical documentation

Version 1.0

Abstract

This document explains every design decision of *AuNW Analysis*. It covers the physically motivated generator of synthetic scanning-electron micrographs, the residual U-Net with three output heads (implemented in Keras 3/TensorFlow), the training strategy against over-fitting, the graph algorithm that separates crossing wires, and the sub-pixel measurement of length and width. It also covers the statistics (including the edge correction for censored lengths), the validation methods and the fine-tuning with real SEM data. All equations correspond line by line to functions in the code; the relevant module is named in each section. The validation results, including failed design alternatives, are reported honestly so that you can judge the limits of the method.

Contents

1 Problem statement and overall design **3**

2 Synthetic SEM images of chaotic AuNW meshes **4**

2.1 Geometry of a single wire 4

2.2 Population model 5

2.3 SEM image formation 5

3 Learning targets and data pipeline **6**

4 Network architecture **7**

4.1 Residual U-Net 7

4.2 Loss functions 8

5 Training and the control of over-fitting **9**

5.1 Optimiser 9

5.2 Countermeasures against over-fitting 9

6 Inference on full SEM images **10**

6.1 Pre-processing 10

6.2 Automatic scale selection 10

6.3 Tiled prediction with seamless blending 10

6.4 Test-time augmentation (TTA) 10

7 Separating individual wires **11**
7.1 Skeleton and junctions 11
7.2 Pairing branches through junctions 11
7.3 End classification and the “complete” flag 12

8 Measurement **12**
8.1 Length 12
8.2 Width with sub-pixel precision 12

9 Statistics **13**

10 Validation **13**

11 Improving the network with real SEM images **16**

12 Using the software **16**

13 Proposed additional features (for discussion) **18**

1 Problem statement and overall design

A gold-nanowire (AuNW) mesh imaged in a scanning electron microscope (SEM) consists of many elongated objects of width $w \approx 10\,\text{nm}$ to $200\,\text{nm}$ and length $L \approx 1\,\mu\text{m}$ to $100\,\mu\text{m}$. They are randomly oriented, overlap at crossings, partially leave the field of view, and are accompanied by by-products such as Au particles. The task is to obtain the *individual* length L_i and width w_i of each wire i, and from them population statistics.

Classical thresholding fails here for two reasons. (i) At a crossing two wires merge into one connected object, so connected-component analysis measures "X"-shaped clusters instead of wires. (ii) SEM contrast is dominated by edge effects and noise, so a global grey-value threshold does not delineate wires reliably. The design therefore splits the problem into a *learned* perception stage and a *deterministic* geometric stage:

1. **Perception (neural network).** A fully convolutional network maps the image $I(\mathbf{x})$ to three dense fields: wire probability $p_m(\mathbf{x})$, centre-line probability $p_c(\mathbf{x})$ and local wire orientation $\theta(\mathbf{x})$.
2. **Geometry (graph algorithm).** The centre-line field is converted into a graph whose edges are wire segments and whose nodes are crossings. Segments are joined through crossings by direction continuity, which yields individual wires.
3. **Measurement and statistics.** Length is measured along the reconstructed centre-line; width from sub-pixel cross-section profiles of p_m. Statistics are then computed on the subset of wires whose length is fully observed.

This hybrid design keeps the parts that need learning (appearance, noise, overlap) inside the network. The parts with an exact mathematical definition (length, width) stay in transparent, testable code. Every step can be inspected in the graphical user interface (GUI).

Why train on synthetic data? Pixel-accurate labels of hundreds of overlapping wires per image are practically impossible to obtain by hand. A renderer that knows the geometry produces unlimited images with *exact* labels, including the true length and width of every wire. That makes it possible to validate the complete measurement chain quantitatively (Section 10). The gap between synthetic and real images is closed by (a) domain randomisation and (b) optional fine-tuning on a few annotated real images (Section 11).

Software architecture. The package `aunw/` contains the science, with no GUI code; `ui/` contains the Streamlit interface. The GUI follows the workflow as numbered steps in a blue navigation column, with the operations of the selected step in the main area (screenshots at the end of this document). This mirrors the "wizard + workspace" pattern of professional image-analysis software. Table 1 gives the mapping.

Table 1: Modules and the workflow step that exposes them in the GUI.

Module	GUI step	Content
`synthetic.py`	1 Synthetic data	wire geometry, SEM image formation, labels
`dataset.py`	1 / 3	dataset files, `tf.data` pipeline, augmentation
`model.py`	2 Network	residual U-Net, losses, metrics, receptive field
`training.py`	3 Training, 8	AdamW, callbacks, live monitor, fine-tuning
`evaluation.py`	4 Validation	Dice/IoU, matched per-wire errors
`io.py`	5 Load images	TIFF/JPEG, normalisation, nm/px from metadata
`inference.py`	6 Analyse	tiling, test-time augmentation, auto-scale
`measure.py`	6 Analyse	skeleton graph, pairing, length, width
`stats.py`, `plots.py`, `export.py`	7 Results	statistics, figures, CSV/Excel/ZIP

2 Synthetic SEM images of chaotic AuNW meshes

2.1 Geometry of a single wire

A wire is modelled as a *capsule*: a centre-line curve γ of length L_c swept by a disc of diameter w. This gives a cylinder of width w with two hemispherical caps, a good model of pentagonally twinned Au nanowires. The true tip-to-tip length is

$$L = L_c + w. \tag{1}$$

The centre-line is a circular arc with constant curvature $\kappa = \beta/L_c$, where β is the total turning angle (bending), drawn uniformly from $[-\beta_{\max}, \beta_{\max}]$. With start point $\mathbf{p}_0$ and start tangent angle θ_0, the tangent angle along arc length $s \in [0, L_c]$ is $\theta(s) = \theta_0 + \kappa s$. Integrating $\dot{\boldsymbol{\gamma}} = (\cos\theta, \sin\theta)$ gives the closed form

$$\boldsymbol{\gamma}(s) = \mathbf{p}_0 + \frac{1}{\kappa}\begin{pmatrix}\sin\theta(s) - \sin\theta_0 \\ \cos\theta_0 - \cos\theta(s)\end{pmatrix}, \qquad |\boldsymbol{\gamma}(s) - \mathbf{c}| = \rho = \frac{1}{|\kappa|}, \quad \mathbf{c} = \mathbf{p}_0 + \rho\,\mathrm{sgn}(\kappa)\begin{pmatrix}-\sin\theta_0 \\ \cos\theta_0\end{pmatrix}. \tag{2}$$

Its length is exactly L_c by construction, which a unit test verifies numerically.

Exact distance field. For every pixel $\mathbf{x}$ we need its distance $d(\mathbf{x})$ to the centre-line and the tangent angle of the closest point. For a circular arc this is analytic. Let $\phi(\mathbf{x})$ be the polar angle of $\mathbf{x} - \mathbf{c}$. The arc parameter of the radial projection is $s^* = L_c/2 + \Delta/\kappa$, with Δ the wrapped difference between $\phi + \mathrm{sgn}(\kappa)\pi/2$ and the mid-arc tangent angle. If $0 \le s^* \le L_c$, then

$$d(\mathbf{x}) = \big|\,|\mathbf{x} - \mathbf{c}| - \rho\,\big|, \qquad \theta(\mathbf{x}) = \phi(\mathbf{x}) + \mathrm{sgn}(\kappa)\tfrac{\pi}{2}; \tag{3}$$

otherwise d is the distance to the nearer end point, and θ the end tangent. (Straight wires, $\kappa \to 0$, use the point–segment distance.) This avoids rasterising lines. Rasterisation would bias the rendered width by up to one pixel, because the envelope of discs centred on jittered pixel centres is wider than the true capsule.

Anti-aliased coverage. The fractional coverage of a pixel is $a(\mathbf{x}) = \mathrm{clip}(w/2 - d(\mathbf{x}) + \frac{1}{2}, 0, 1)$. Integrating a across the wire gives exactly w, so the rendered width is unbiased to sub-pixel precision.

2.2 Population model

Per image, a median length m, a log-normal shape σ_L, a mean width $\bar{w}$ and a width coefficient of variation are drawn. The wires of that image then follow

$$L \sim \mathrm{LogNormal}(\ln m, \sigma_L^2), \qquad w \sim \mathcal{N}(\bar{w}, (\mathrm{CV}\,\bar{w})^2), \tag{4}$$

with uniformly random orientation $\theta_0 \in [0, 2\pi)$ and mid-points uniform over an area 30 % larger than the image, so that wires enter and leave the field of view. The number of wires per 256^2 tile is uniform in $[6, 45]$, which ranges from sparse meshes to very dense chaotic ones. The list order is the z-order: later wires lie on top.

2.3 SEM image formation

Edge contrast. The secondary-electron (SE) yield of a surface element tilted by φ against the beam follows approximately the secant law $\delta(\varphi) \approx \delta_0 / \cos\varphi$, because the escape-depth volume intersected by the beam grows as $1/\cos\varphi$. A cylinder of radius $R = w/2$, at lateral offset r from its axis, has height $z = \sqrt{R^2 - r^2}$, slope $\mathrm{d}z/\mathrm{d}r = -r/z$, and therefore

$$\cos\varphi = \frac{1}{\sqrt{1 + (\mathrm{d}z/\mathrm{d}r)^2}} = \sqrt{1 - (r/R)^2} \quad \Rightarrow \quad I_{\mathrm{wire}}(r) = I_w \Big[1 + g\Big(\frac{1}{\sqrt{1 - (r/R)^2}} - 1\Big)\Big], \tag{5}$$

with a random edge gain $g \in [0.15, 0.6]$ and $|r/R|$ clipped at 0.97 (Fig. 1). This produces the bright wire rims typical of SE images. A weak exponential shadow halo $1 - h\,e^{-(d-R)/\lambda}$ outside the wire models detector shadowing.

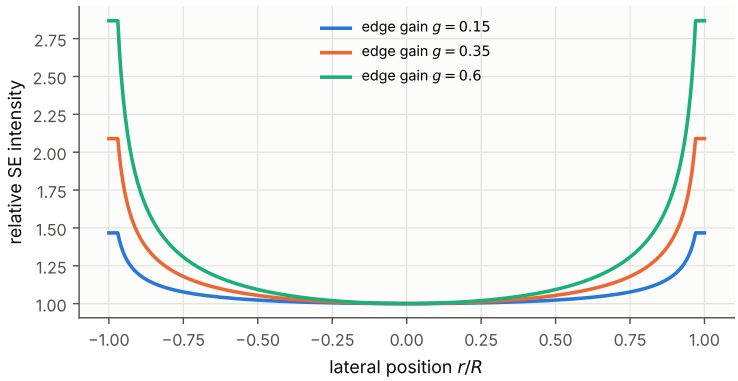


Figure 1: Cross-section intensity of a wire for three edge gains g (secant law on a cylinder).

Substrate and distractors. The background is a random grey level with low-frequency fluctuations (Gaussian-filtered noise), fine grain and a linear charging gradient. With probability 0.25 dark pores of a filter membrane are added. With probability 0.5 Au nanoparticles are added. They are rendered with the same edge physics but are *not* labelled as wires, so the network must learn to ignore them.

Acquisition. With the ideal image J, the recorded image is

$$I = \Big[\tfrac{1}{N}\,\mathrm{Poisson}\big(N\,(J * G_{\sigma_b})\big) + \varepsilon_{\mathrm{read}} + \eta_{\mathrm{row}}\Big]^{\gamma}, \tag{6}$$

where G_{σ_b} is the probe point-spread function (Gaussian, $\sigma_b \in [0.4, 1.4]$ px). $N \in [15, 250]$ is the electron dose per unit intensity, which gives a signal-to-noise ratio $\mathrm{SNR} = \sqrt{NJ}$. $\varepsilon_{\mathrm{read}}$ is Gaussian read noise, η_{row} a random offset per scan line, and $\gamma \in [0.8, 1.25]$ a detector non-linearity.

With probability 0.3 the image is JPEG re-compressed at quality 40–95, since colleagues often save micrographs as JPEG. Finally, the same robust normalisation as for real images is applied (Section 6.1).

Domain randomisation. Every one of these parameters is drawn anew for every image. The network therefore sees a *distribution* of imaging conditions that contains real microscopes as plausible samples, and it is forced to learn the invariant: the shape of a wire. Figure 2 shows examples with their labels.

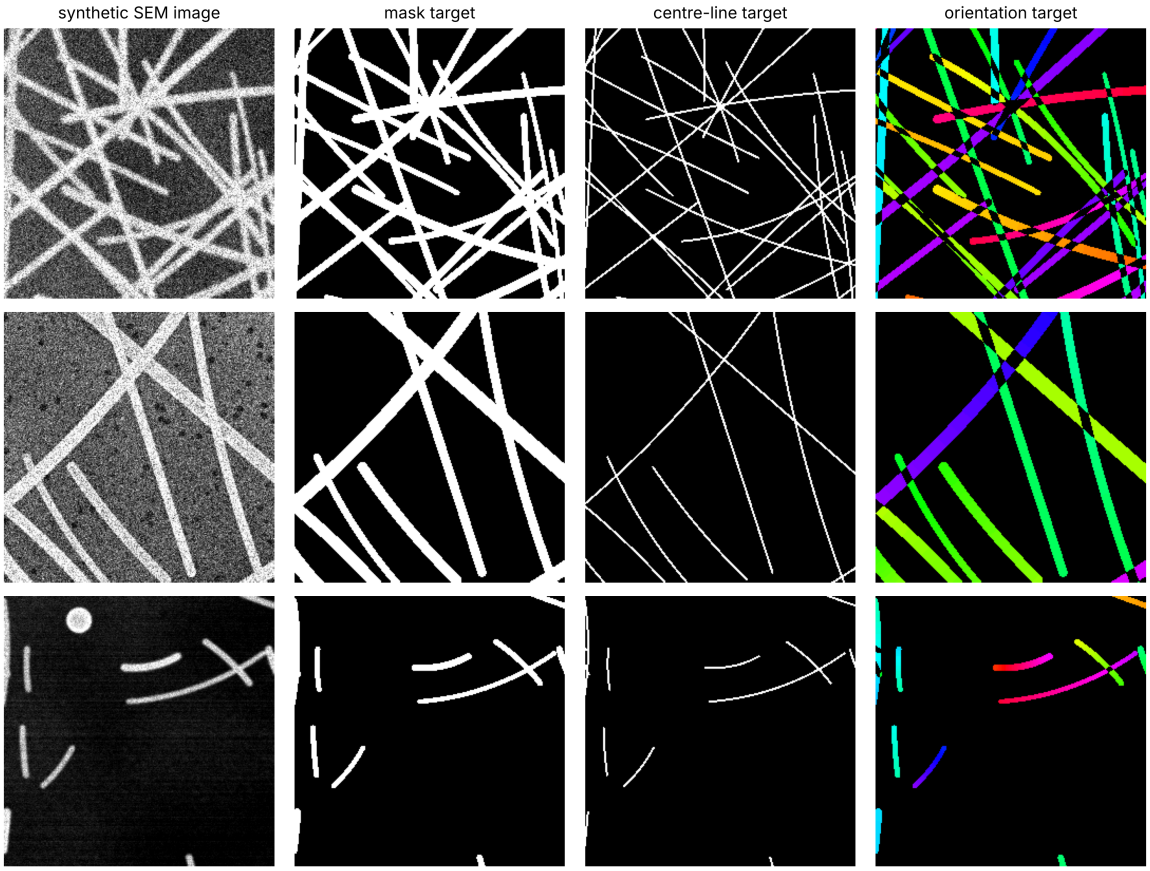


Figure 2: Synthetic training tiles and their exact labels. Orientation is colour-coded by hue; black marks pixels without a reliable orientation (background and crossings).

3 Learning targets and data pipeline

For every training tile three targets are stored (`dataset.py`):

Mask $y_m(\mathbf{x}) = 1$ if any wire covers $\mathbf{x}$ ($a \geq \frac{1}{2}$).

Centre-line $y_c(\mathbf{x}) = 1$ if $d(\mathbf{x}) \leq 1\,\text{px}$ for any wire, a 3-px wide line. At a crossing the two centre-lines cross cleanly, unlike the mask, where the wires merge into a blob.

Orientation The wire axis is an *axial* quantity: θ and $\theta + \pi$ describe the same wire. Regressing θ directly would create an artificial discontinuity at $0°/180°$. We therefore use the double-angle representation

$$\mathbf{o} = (\cos 2\theta, \sin 2\theta) \in S^1, \tag{7}$$

which is continuous and unique for axial data. The prediction $\hat{\mathbf{o}}$ gives $\hat{\theta} = \frac{1}{2}\,\text{atan2}(\hat{o}_2, \hat{o}_1)$ and a

confidence $|\hat{\mathbf{o}}|$. The squared error relates to the angle error $\Delta\theta$ by

$$\|\mathbf{o} - \hat{\mathbf{o}}\|^2 = 2 - 2\cos 2\Delta\theta = 4\sin^2 \Delta\theta \approx 4\Delta\theta^2. \tag{8}$$

At pixels covered by two wires the orientation is ambiguous, so a weight $\mu(\mathbf{x}) = 0$ excludes them (and the background) from the loss.

Label-consistent augmentation. Random flips and 90° rotations multiply the effective data by 8. They must be applied to the orientation *vector*, not only to the pixel grid. A horizontal or vertical flip maps $\theta \mapsto -\theta \pmod{\pi}$, hence $(o_1, o_2) \mapsto (o_1, -o_2)$. A rotation by 90° maps $\theta \mapsto \theta + \pi/2$, hence $2\theta \mapsto 2\theta + \pi$ and $(o_1, o_2) \mapsto (-o_1, -o_2)$. These sign rules are implemented in `_augment` and verified by a unit test that compares the transformed label with the principal axis of the transformed mask. Photometric augmentation (contrast, brightness, gamma, Gaussian noise) acts only on the input.

Pipeline. `tf.data.Dataset.from_tensor_slices` → `map` (targets) → `shuffle` → `map` (augmentation, parallel) → `batch` → `prefetch`. Augmentation runs on the CPU in parallel with the training step on the accelerator. The validation set is rendered with a different random seed and is never augmented.

4 Network architecture

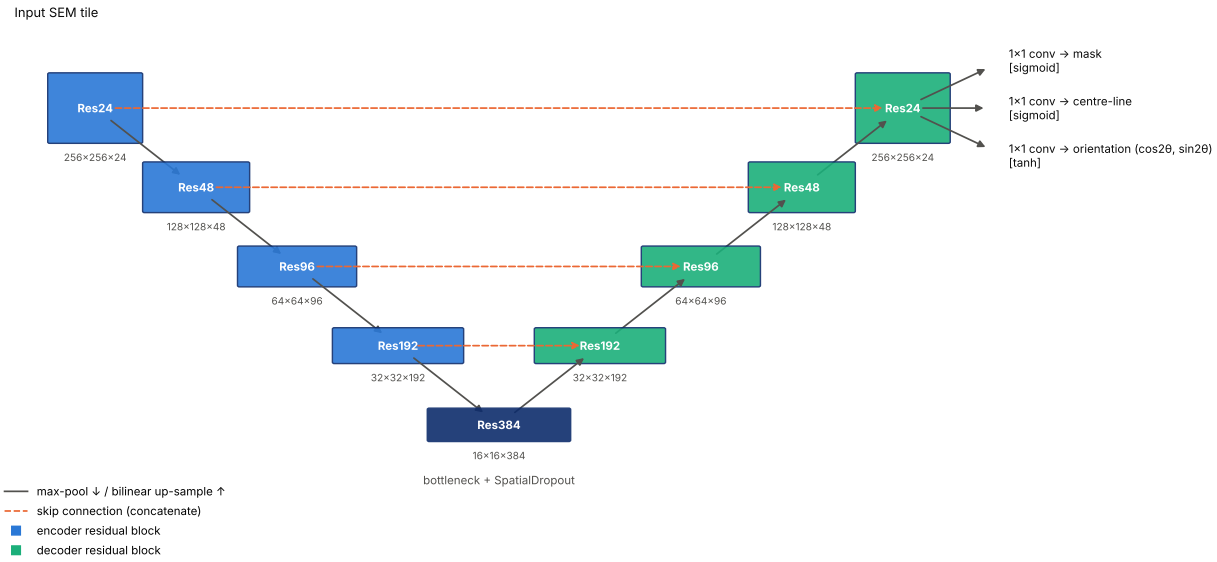


Figure 3: Residual U-Net (default configuration: base 24 filters, depth 4). Box height ∝ resolution, label = output shape.

4.1 Residual U-Net

The U-Net [1] is the standard architecture for dense prediction when training data are limited. An *encoder* halves the resolution four times while doubling the channels ($24 \to 48 \to 96 \to 192 \to 384$), so it learns increasingly abstract and spatially extended features. A *decoder* restores full resolution. *Skip connections* concatenate encoder features of equal resolution into the decoder, restoring the precise edge positions lost by pooling, which is essential for measuring widths. Each block is a residual block [2]:

$$\mathbf{y} = \mathrm{ReLU}\Big(\mathrm{BN}\big(W_2 * \mathrm{ReLU}(\mathrm{BN}(W_1 * \mathbf{x}))\big) + P\mathbf{x}\Big), \tag{9}$$

with 3×3 convolutions $W_{1,2}$ and P either the identity or a 1×1 projection when the channel count changes. The identity path gives gradients a short route through the network and eases optimisation. Batch normalisation, $\mathrm{BN}(x) = \gamma\,(x - \mu_B)/\sqrt{\sigma_B^2 + \epsilon} + \beta$, stabilises training and acts as a mild regulariser. Convolutions before BN carry no bias, because BN's β makes it redundant. A convolution has $k^2 C_{\mathrm{in}} C_{\mathrm{out}}$ weights; the default network has 4.67 M parameters, and the CPU preset (16 base filters) has 2.08 M.

Up-sampling. Bilinear up-sampling followed by convolution is used instead of transposed convolutions. Transposed convolutions with stride 2 produce checkerboard artefacts [3], a periodic bias of edge positions that would translate directly into width errors.

Receptive field. With kernel size k and cumulative stride ("jump") j, each layer enlarges the receptive field by $r \leftarrow r + (k-1)j$, and pooling multiplies the jump: $j \leftarrow 2j$. Summing over the encoder, bottleneck and decoder (`receptive_field`) gives

$$r = 1 + \sum_{\ell=0}^{3} (4 \cdot 2^{\ell} + 2^{\ell}) + 4 \cdot 16 + \sum_{\ell=0}^{3} 4 \cdot 2^{\ell} = 200 \text{ px}. \tag{10}$$

At the target wire width of about 9 px (Section 6.2), one output pixel therefore sees more than 20 wire widths. That is enough context to see both arms of a crossing and to decide which centre-line continues where.

Three heads. Three 1×1 convolutions on the last decoder features give $p_m = \sigma(\cdot)$, $p_c = \sigma(\cdot)$ and $\hat{\mathbf{o}} = \tanh(\cdot) \in (-1, 1)^2$. The heads share all features (multi-task learning). The orientation task forces the shared features to encode direction, which also helps the centre-line head at crossings. The heads are computed in float32 even under mixed precision.

4.2 Loss functions

For a binary map with targets y_i and predictions p_i over the N pixels of a tile,

$$\mathcal{L}_{\mathrm{BCE}} = -\frac{1}{N} \sum_i \left[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \right], \tag{11}$$

$$\mathcal{L}_{\mathrm{Dice}} = 1 - \frac{2 \sum_i y_i p_i + \epsilon}{\sum_i y_i + \sum_i p_i + \epsilon}, \tag{12}$$

and the mask and centre-line heads use $\mathcal{L}_{\mathrm{BCE}} + \mathcal{L}_{\mathrm{Dice}}$. BCE yields calibrated probabilities, which matters because the width is measured at $p_m = \frac{1}{2}$. The Dice term is invariant to class imbalance; the centre-lines cover only a few percent of the pixels, and with BCE alone the trivial prediction "no centre-line" would already have a low loss. The orientation loss is the weighted squared error

$$\mathcal{L}_{\mathrm{o}} = \frac{\sum_i \mu_i \, \| \mathbf{o}_i - \hat{\mathbf{o}}_i \|^2}{\sum_i \mu_i + 1}, \tag{13}$$

and the total loss is $\mathcal{L} = \mathcal{L}_m + \mathcal{L}_c + 0.5\,\mathcal{L}_o$. Monitored metrics are the hard Dice coefficient (F1 score) at threshold 0.5 for mask and centre-line, and the mean axial angle error $\overline{\min(|\Delta\theta|, \pi - |\Delta\theta|)}$.

5 Training and the control of over-fitting

5.1 Optimiser

AdamW [4] with gradient g_t:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \tag{14}$$

$$\theta_t = \theta_{t-1} - \eta_t \Big(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda\, \theta_{t-1} \Big), \quad \hat{m}_t = \frac{m_t}{1-\beta_1^t},\ \hat{v}_t = \frac{v_t}{1-\beta_2^t}. \tag{15}$$

The weight decay $\lambda = 10^{-4}$ is *decoupled* from the adaptive gradient scaling. Classical L_2 regularisation added to the loss would be divided by $\sqrt{\hat{v}_t}$, so parameters with large gradients would be regularised less, which is the opposite of the intent. Gradients are clipped to norm 1 for robustness against rare outlier batches. The start learning rate is $\eta_0 = 10^{-3}$.

5.2 Countermeasures against over-fitting

Over-fitting means that the training loss keeps decreasing while the loss on unseen data rises: the network memorises the training samples instead of the underlying regularity. The following measures act at different levels:

1. **Independent validation set** from a different generator seed (and, for real data, from *different images*; Section 11). This is the only honest estimate of the generalisation error.
2. **Early stopping** with restoration of the best weights: with $t^* = \arg\min_{t' \le t} \mathcal{L}_{\text{val}}(t')$, training stops when $t - t^* \ge P$ ($P = 10$), and the weights of epoch t^* are restored. This is equivalent to choosing the training length by validation.
3. **Learning-rate reduction** on a plateau: $\eta \leftarrow \eta/2$ after 4 epochs without improvement. A smaller step size often resolves an apparent plateau and gives a better minimum before early stopping triggers.
4. **Capacity regularisation**: decoupled weight decay, `SpatialDropout2D` (rate 0.15) in the two deepest levels and the bottleneck, where most parameters are, and BatchNorm. SpatialDropout drops whole feature maps rather than single activations, because neighbouring activations in a conv layer are strongly correlated and ordinary dropout would hardly regularise them.
5. **Data augmentation** (Section 3) and the practically unlimited synthetic data.

Training as a separate process. The GUI does not train inside its own process. It launches the same command a user could type (`python -m aunw.cli train ...`) as a separate process, which writes its progress to `progress.json` and `preview.npz` in the model folder; the GUI only reads these files. A stop request is a file `STOP`, which the trainer checks after every batch, keeping the best weights. This design means that a crash in the numerical libraries can never take down the user interface (an early threaded version did exactly that inside the web server). It also means that training continues when the browser is closed, and that every GUI run is reproducible from the command shown on screen.

Monitoring. The GUI (step 3) and TensorBoard show the per-batch loss, all per-epoch metrics for training and validation, the learning rate, and predictions on fixed validation tiles after every epoch. In addition, `overfitting_report` computes the generalisation gap

$$\Delta_t = \mathcal{L}_{\text{val}}(t) - \mathcal{L}_{\text{train}}(t) \tag{16}$$

and raises an over-fitting flag when $\mathcal{L}_{\text{val}}$ has risen over the last 3 epochs while $\mathcal{L}_{\text{train}}$ fell. Note that Δ_t can be negative early in training: training metrics are averaged over an epoch with dropout

active and with augmented samples, whereas validation is evaluated at the end of the epoch, in inference mode, on unaugmented data.

6 Inference on full SEM images

6.1 Pre-processing

Images are converted to grey (luminance for RGB) and normalised robustly: $I \leftarrow \mathrm{clip}((I - q_{0.5})/(q_{99.5} - q_{0.5}), 0, 1)$, with q_p the p-th percentile. Percentiles instead of minimum/maximum make the result insensitive to hot pixels and to the 16-bit range of many SEM TIFFs. The instrument information bar is detected as a block of rows that are mostly saturated, then cropped; the value can be corrected in the GUI. The pixel size is read from FEI/Thermo Fisher metadata (`PixelWidth`, in metres), Zeiss SmartSEM metadata (`Image Pixel Size`) or ImageJ calibration. It can also be entered by hand or computed from the scale bar as $s = \ell_{\mathrm{bar}}[\mathrm{nm}]/\ell_{\mathrm{bar}}[\mathrm{px}]$.

6.2 Automatic scale selection

The network was trained on wires $\approx$ 3–20 px wide. A real micrograph is therefore resized by a factor s so that the median wire width is close to the target $w^* = 9\,\mathrm{px}$. Starting at $s_0 = 1$, a central crop is analysed and the median width $\hat{w}(s_n)$ measured; then

$$s_{n+1} = s_n \, \frac{w^*}{\hat{w}(s_n)}, \tag{17}$$

until $w^*/\hat{w} \in (0.75, 1.33)$ (at most 3 iterations). Iteration is necessary because a network that sees wires far outside its training range under-estimates how wrong the scale is. All measurements are divided by s afterwards.

6.3 Tiled prediction with seamless blending

The image is processed in tiles of $T = 256\,\mathrm{px}$ with overlap $o = 64\,\mathrm{px}$ (stride $T - o$). After reflective padding by o on every side, the number of tiles along an axis of length H is $n = \lceil (H + 2o - T)/(T - o) \rceil + 1$. Each tile prediction $f(T_k)$ is weighted by a window $\omega(\mathbf{x}) = \omega_1(x)\omega_1(y)$ that is 1 in the centre and decays as $\sin^2$ over the overlap. The blended prediction is

$$P(\mathbf{x}) = \frac{\sum_k \omega_k(\mathbf{x})\, f(T_k)(\mathbf{x})}{\sum_k \omega_k(\mathbf{x})}. \tag{18}$$

Tile borders, where convolutions see artificial padding, receive almost zero weight, so no seams appear. A unit test with an identity network verifies $P \equiv I$ to 10^{-5} for arbitrary image sizes.

6.4 Test-time augmentation (TTA)

With the flip group $\mathcal{G} = \{\mathrm{id}, \mathrm{LR}, \mathrm{UD}, \mathrm{LR} \circ \mathrm{UD}\}$, the prediction is $\bar{P} = \frac{1}{4} \sum_{g \in \mathcal{G}} g^{-1} f(gI)$. Mask and centre-line maps are simply flipped back. For the orientation, a single flip reverses the sign of o_2, while the double flip (a 180° rotation) leaves $\mathbf{o}$ unchanged. Averaging double-angle vectors is the correct axial mean. TTA reduces the variance of the prediction at four times the cost.

7 Separating individual wires

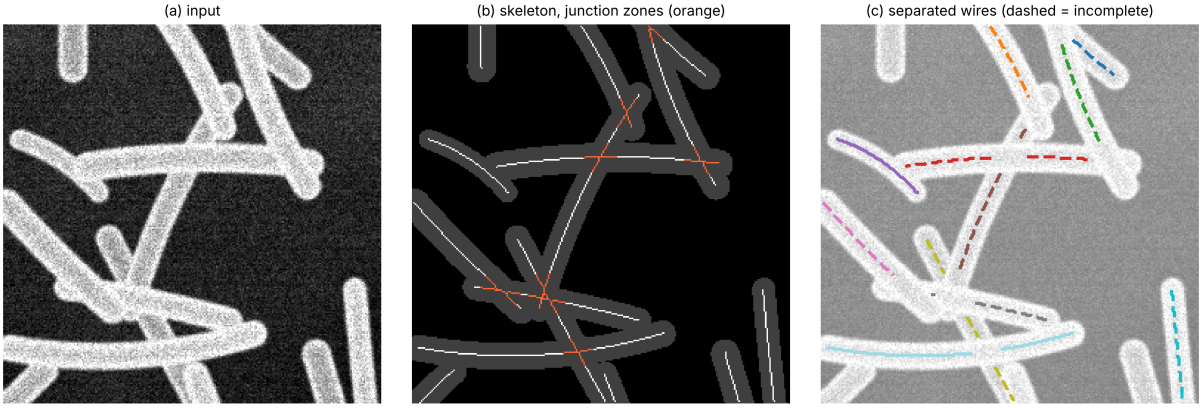


Figure 4: Wire separation on a synthetic tile. (b) Skeleton of the centre-line map with junction zones. (c) Wires after pairing branches through junctions; each colour is one physical wire.

7.1 Skeleton and junctions

The centre-line map is thresholded inside the mask, $C = \{p_c > \frac{1}{2}\} \cap \{p_m > \frac{1}{2}\}$, and thinned to an 8-connected skeleton S of one-pixel width. For a skeleton pixel with cyclically ordered neighbours $P_1, \ldots, P_8$ ($P_9 = P_1$), the *crossing number*

$$\mathrm{CN} = \tfrac{1}{2} \sum_{k=1}^{8} |P_{k+1} - P_k| \tag{19}$$

counts the number of branches leaving the pixel: 1 at an end, 2 on a line, ≥ 3 at a junction. Unlike the neighbour count, CN does not create false junctions on staircase-shaped diagonal lines. Compact junction clusters, where every pixel individually has CN = 2, are caught by an additional neighbour count ≥ 4. Validation showed that 4 % of skeleton pixels were otherwise traced incorrectly. Junction pixels are dilated into *junction zones* of radius $r_j = \lceil w/2 \rceil + 1$ (with w the global width estimate $w \approx 2\,\mathrm{EDT} - 1$ on the skeleton). Removing the zones splits the skeleton into *branches*, simple pixel chains. Each chain is ordered by walking from an end in both directions.

7.2 Pairing branches through junctions

For every branch end inside junction j we compute an outward unit vector $\mathbf{u}$ pointing into the junction. Its axis comes from the axial mean of the orientation head over the last $m = \max(8, 3w)$ pixels, when that is confident ($|\bar{\mathbf{o}}| > 0.5$); its sign comes from the geometry. Two ends a, b with tip separation $\mathbf{r}$ are candidates for one wire running straight through the junction if

$$\delta_{ab} = \arccos(-\mathbf{u}_a \cdot \mathbf{u}_b) < \delta_{\max} = 25^\circ \quad \text{(deflection)}, \tag{20}$$

$$\ell_{ab} = \max(|\mathbf{u}_a \times \mathbf{r}|, |\mathbf{u}_b \times \mathbf{r}|) < \tau(|\mathbf{r}|) = \max(2, \tfrac{w}{2}) + 0.1|\mathbf{r}| \quad \text{(collinearity)}. \tag{21}$$

The tolerance grows with the gap because a direction error ϵ produces a lateral offset $\approx \epsilon|\mathbf{r}|$. Candidates are accepted greedily in order of increasing cost $c_{ab} = \delta_{ab}/\delta_{\max} + \ell_{ab}/\tau$, with every end used at most once and no cycles. Accepted pairs are merged with a union–find structure; each resulting set of branches is one wire. The tolerance model was not guessed. It was chosen from logged statistics of true and false pairs on ground-truth maps: true continuations have a median lateral offset of 0.9 px, false pairs 9 px, and the fixed 2-px tolerance used initially rejected 22 % of true pairs.

Gap bridging. Where two wires cross, the centre-line of the *lower* wire can be interrupted, because the upper wire hides it. Its skeleton then ends in two facing free tips instead of a junction. Two free tips are therefore also linked when they are at most $2.5w + 3\,\mathrm{px}$ apart (candidates from a k-d tree), satisfy the same deflection and collinearity criteria, point towards each other ($\mathbf{u}_a \cdot \mathbf{r} > 0$), and the straight segment between them lies entirely inside the wire mask ($\min p_m > \frac{1}{2}$ along it), i.e. the wire visibly continues underneath.

7.3 End classification and the "complete" flag

Every unpaired end is classified as a *tip* (a free end not at the border), a *T-end* (the only unpaired end in its junction: all other arms continue, so the wire ends on top of another wire), *unresolved* (unpaired in a junction with other unpaired ends), or *border*. A wire is **complete** if both ends are tips and it does not touch the border. Only then is its length fully observed. On ground truth, complete wires had a gross length error ($< -10\,\%$) in only $0.5\,\%$ of cases, against $26\,\%$ for wires with one T-end and $62\,\%$ with two. Length statistics therefore use complete wires; width statistics use all wires, because truncation does not affect width.

8 Measurement

8.1 Length

A digital straight line at angle $\varphi \in [0°, 45°]$ consists of $n_e = L(\cos\varphi - \sin\varphi)$ axial and $n_o = L\sin\varphi$ diagonal steps. The naive chain length is therefore

$$L_{\mathrm{chain}} = n_e + \sqrt{2}\, n_o = L\big(\cos\varphi + (\sqrt{2} - 1)\sin\varphi\big), \tag{22}$$

which over-estimates L by up to $\sqrt{4 - 2\sqrt{2}} - 1 = 8.2\,\%$ at $\varphi = 22.5°$. We instead resample the ordered chain every $k = 4$ pixels and sum the Euclidean polyline. With independent quantisation errors $\varepsilon \sim \mathcal{U}(-\frac{1}{2}, \frac{1}{2})$ perpendicular to the line, $\mathrm{Var}(\Delta\varepsilon) = \frac{1}{6}$ and

$$\frac{\mathbb{E}[L_{\mathrm{poly}}] - L}{L} \approx \frac{\mathbb{E}[(\Delta\varepsilon)^2]}{2k^2} = \frac{1}{12k^2} = 0.5\,\% \quad (k = 4). \tag{23}$$

The wire length is the sum of its branch lengths, the bridged junction gaps $|\mathbf{r}_{ab}|$, and end corrections. Each free end adds $w/2$, because the centre-line stops at the centre of the hemispherical cap, cf. Eq. (1). An end inside a junction adds the distance to the junction centre, bounded by $r_j + w$.

8.2 Width with sub-pixel precision

At every second centre-line pixel, at least one width away from tips and junctions, the mask probability is sampled along the local normal $\mathbf{n}$ (bilinear interpolation, step $\Delta t = 0.25\,\mathrm{px}$): $p(t) = p_m(\mathbf{x} + t\mathbf{n})$. The edges are the first samples with $p \le \frac{1}{2}$ on either side, refined by linear interpolation:

$$t_{\mathrm{edge}} = t_{k-1} + \frac{p_{k-1} - \frac{1}{2}}{p_{k-1} - p_k}\,\Delta t, \qquad w_i = t^{+}_{\mathrm{edge}} + t^{-}_{\mathrm{edge}}. \tag{24}$$

The $\frac{1}{2}$ level is the natural edge criterion: a step edge convolved with any symmetric point-spread function crosses $\frac{1}{2}$ exactly at the true edge, and a calibrated posterior equals $\frac{1}{2}$ at the decision boundary. The wire width is the *median* of all w_i, which is robust to the outliers caused by touching neighbours and particles. If fewer than 3 profiles are valid, the distance-transform estimate $2\,\mathrm{EDT} - 1$ is used.

9 Statistics

For a sample $x_1, \ldots, x_n$ the software reports the mean $\bar{x}$, the standard deviation s, the SEM $s/\sqrt{n}$ and the 95 % confidence interval $\bar{x} \pm t_{0.975,n-1}\, s/\sqrt{n}$. It also reports the median with a bootstrap 95 % interval (2.5 and 97.5 percentiles of the median over $B = 2000$ resamples), the quartiles and the coefficient of variation $100\, s/\bar{x}$.

Log-normal model. Nanowire lengths are typically right-skewed and well described by $\ln X \sim \mathcal{N}(\mu, \sigma^2)$. The maximum-likelihood estimates are $\hat{\mu} = \overline{\ln x}$ and $\hat{\sigma}^2 = \overline{(\ln x - \hat{\mu})^2}$, with median $e^{\hat{\mu}}$, mean $e^{\hat{\mu}+\hat{\sigma}^2/2}$, geometric mean $e^{\hat{\mu}}$ and geometric SD $e^{\hat{\sigma}}$. Normal and log-normal models are compared with the Akaike information criterion, $\mathrm{AIC} = 2k - 2\ln\hat{\mathcal{L}}$ (with $k = 2$ for both); the model with the lower AIC is reported. Histograms use the Freedman–Diaconis bin width $h = 2\,\mathrm{IQR}\, n^{-1/3}$.

Edge correction (Miles–Lantuéjoul). A wire with bounding box $\Delta x \times \Delta y$, placed uniformly at random, lies completely inside a $W \times H$ frame with probability

$$\pi_i = \frac{(W - \Delta x_i)(H - \Delta y_i)}{WH}. \tag{25}$$

Long wires are therefore under-represented among complete wires. Weighting each complete wire by $1/\pi_i$ (Horvitz–Thompson/Hájek estimator, $\bar{x}_w = \sum w_i x_i / \sum w_i$) removes this bias [5, 6]. The weighted standard error uses the Kish effective sample size $n_{\mathrm{eff}} = (\sum w_i)^2 / \sum w_i^2$.

Stereological mean length. An independent estimate of the mean length needs no wire separation at all. Consider a stationary random system of wires with number density λ (wires per area) and mean length $\langle L \rangle$. A window of area A contains on average $2\lambda A$ wire ends and a total wire length of $\lambda A \langle L \rangle$, irrespective of how many wires are cut by the frame. Hence

$$\widehat{\langle L \rangle} = \frac{2\,\Lambda}{N_{\mathrm{ends}}}, \tag{26}$$

with Λ the total measured wire length in the window and N_{ends} the number of visible ends (free tips and T-ends; ends caused by unresolved crossings and border ends are excluded). The estimator is insensitive to pairing errors, because splitting a wire changes neither Λ nor the number of true ends. It is reported per image next to the mean of the complete wires; agreement of the two is a practical consistency check.

Reliability classes by areal coverage. The fraction ϕ of the image covered by wires determines how many wire ends are hidden under other wires. Each image is classified as *good* ($\phi \le 0.30$), *caution* ($0.30 < \phi \le 0.45$) or *length unreliable* ($\phi > 0.45$). Width statistics stay reliable in all classes (Section 10).

Comparing populations. To compare automatic and manual measurements, or two samples, the software reports the two-sample Kolmogorov–Smirnov test $D = \sup_x |F_1(x) - F_2(x)|$ (shape), the Mann–Whitney U test (location, non-parametric) and Welch's t test (means, unequal variances), together with the relative differences of means and medians.

10 Validation

Two levels are validated separately, because they fail for different reasons.

(1) The measurement algorithm alone. The graph and measurement stages were run on *perfect* maps, i.e. the ground-truth labels, for eight dense synthetic 768^2 images, about 130–400 wires each. These numbers are the best the algorithm can do, independent of the network:

Complete wires: median relative length error	+0.7 %
Complete wires: length error within ±10 % / ±20 %	91 % / 95 %
All matched wires: median relative width error	+0.65 %
Single straight wire, four angles (unit test)	length < 3 %, width < 5 %

(2) The complete chain with the trained network. The network shipped with the software (`models/cpu_starter_f16`: 16 base filters, 2.1 M parameters) was trained on 1600 synthetic tiles for 35 epochs on 4 CPU cores (about 2 h). Training and validation loss decreased together (Fig. 5): there was no over-fitting, and the best validation loss was reached in the last epoch. A longer run on a GPU, or the larger default network (24 filters), can therefore be expected to improve the results further.

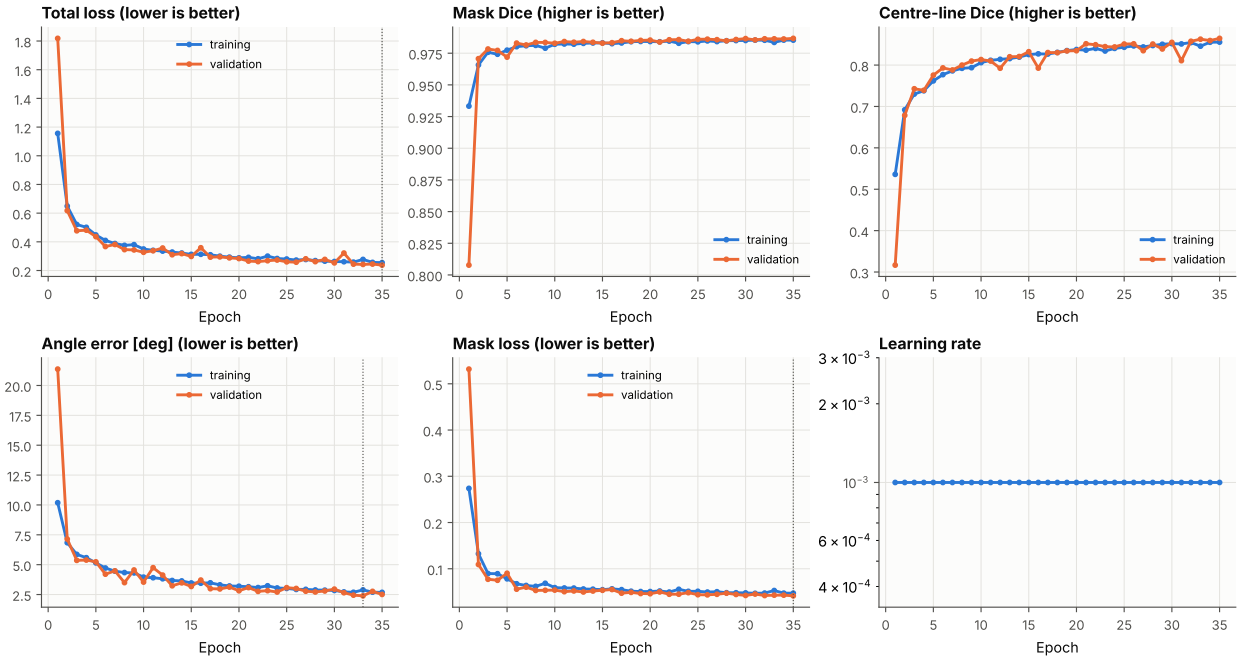


Figure 5: Training of the shipped network: training (blue) and validation (orange) curves. The validation loss follows the training loss closely, so there is no generalisation gap.

The complete pipeline (tiling, TTA, wire separation, measurement) was evaluated on two independent sets of full-size 768^2 test images. Set A has the full density range of the training generator (coverage 7–80 %); set B has realistic moderate densities (coverage 4–44 %).

Table 2: Accuracy of the complete chain. Per-wire errors refer to wires matched to the ground truth; length errors to *complete* wires. Per-image biases compare the statistics a user would report for one image with the truth (median over images of the absolute bias).

	A: all	A: coverage $\leq 30\,\%$	B: moderate
Images	12	5	16
Mask Dice / IoU	0.985 / 0.970	–	0.982 / 0.966
Centre-line Dice	0.859	–	0.924
Orientation error	2.7°	–	1.8°
Crossings resolved	82 %	–	93 %
Length: median rel. error (per wire)	+0.07 %	+0.24 %	+0.37 %
Length: within ±10 %	85 %	92 %	92 %
Width: median rel. error (per wire)	−0.46 %	−0.29 %	−0.70 %
Width: within ±10 %	77 %	86 %	92 %
Per image: |bias| of median length	20 %	9.0 %	4.4 %
Per image: |bias| of stereological $\langle L \rangle$	29 %	9.1 %	10 %
Per image: |bias| of median width	2.5 %	3.3 %	1.2 %

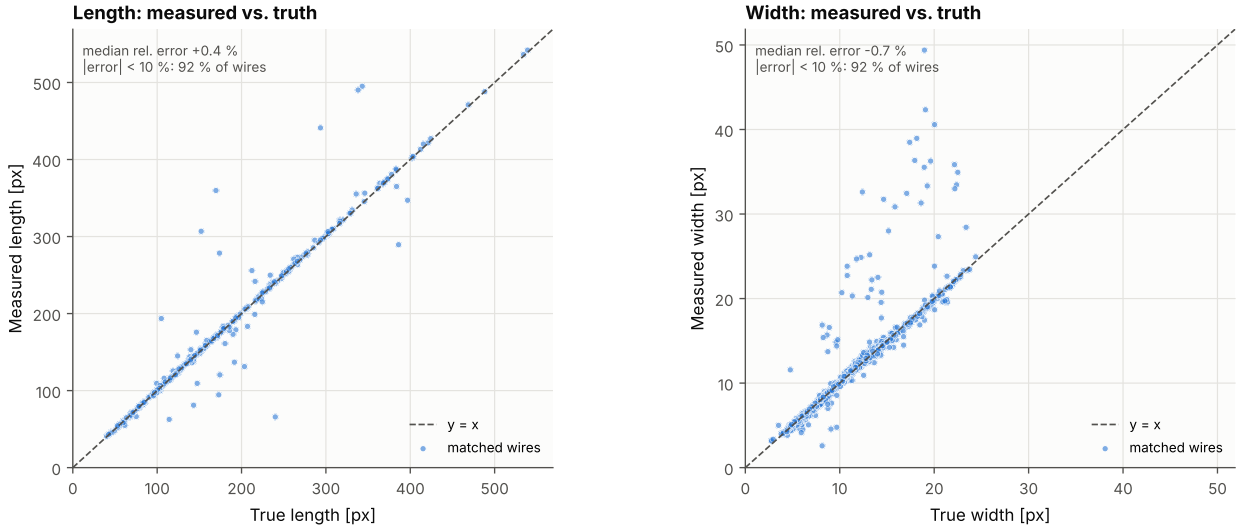


Figure 6: Measured vs. true length (complete wires) and width (all matched wires) on test set B.

Interpretation. Individual wires are measured essentially without bias: median errors are below 1 % for both length and width. At moderate density, the per-image deviations of the median length (typically 4 %, at most 14 %) are consistent with pure sampling noise. With $n \approx 20$ complete wires from a log-normal population with $\sigma \approx 0.45$, the standard error of the median is $\approx 1.25\,\sigma/\sqrt{n} \approx 12\,\%$. This is why the software always reports bootstrap confidence intervals, and why several images per sample should be analysed.

The physical limit: dense meshes. Above roughly 40 % areal coverage most wire ends lie underneath other wires and cannot be observed, by this software or by a human. Complete wires then become rare (fewer than 35 in a 768^2 image) and preferentially short, because a long wire crosses more wires and is more likely to lose an end ($P(\text{complete}) \propto p^k$ for k crossings resolved with probability p). The stereological estimator also degrades, because hidden ends are not counted. The width remains reliable at all densities (per-image bias $\leq 5\,\%$ up to 80 % coverage). The software therefore reports the coverage and a reliability class for every image (Section 9). For length measurements on dense meshes, image regions of lower density or higher magnification should be used.

Design alternatives that were tested and rejected. Reporting these is part of good scientific practice:

- *Junction contraction* (merging junctions joined by short branches, intended for shallow-angle crossings) reduced the fraction of wires within ±10 % from 71 % to 60 % and was removed.
- *Counting T-ends as complete* raised the number of usable wires but admitted 26–62 % fragments.
- *Gap bridging* (Section 7) was kept although its measurable gain on the test sets was small (crossings resolved 79 %→82 %); the dominant length bias in dense meshes is the physical selection effect above, not interrupted centre-lines.
- *Horvitz–Thompson weights* p^{-k} for the survivorship bias, with p estimated per image (globally or per junction degree), over-corrected the mean by +36 to +57 % because p is strongly heterogeneous, and was not adopted.

11 Improving the network with real SEM images

The synthetic renderer is an approximation of reality, so a residual *domain gap* is expected. The software provides a closed improvement loop:

1. Analyse a real micrograph (step 6) and download the predicted mask.
2. Correct the mask in ImageJ/Fiji or GIMP (white = wire). Correcting is much faster than drawing from scratch.
3. Upload image and mask pairs in step 8. Images are rescaled with the same automatic scale as in the analysis, then cut into tiles. The centre-line target is derived from the skeleton of the mask; the orientation weight is set to zero, since real data teach mask and centre-line, while synthetic data keep teaching orientation.
4. Fine-tune with the **encoder frozen** (general features are kept and only the decoder adapts), a small learning rate (10^{-4}), and every batch a fixed mixture of real and synthetic tiles (default 50/50). Mixing prevents *catastrophic forgetting* of the synthetic knowledge. Early stopping monitors the loss on *held-out real images*.
5. Compare old and new networks in step 4: Dice on annotated real images, or agreement with manual measurements via the KS test.

The train/validation split for real data must be made *by image, never by tile*. Neighbouring tiles of one micrograph share wires, illumination and noise, so a tile-wise split would leak information and the validation loss would be over-optimistic.

12 Using the software

Install with `pip install -r requirements.txt` and start with `streamlit run app.py`. The blue column on the left lists the steps; completed steps are marked with ✓. The active network is selected at the bottom of the column. For routine analysis only steps 5–7 are needed:

Step 5 Upload TIFF/JPEG/PNG files, or read a whole folder. Check the cropped information bar and the calibration (metadata, nm/px or scale bar). The calibration of the first image can be applied to all.

Step 6 Run the analysis. The overlay colours complete wires blue, wires ending in a crossing orange, and wires cut by the border grey. Tabs show the raw network output and the table of all wires.

Step 7 Choose the unit and the edge correction. The page shows the summary table, histograms with normal and log-normal fits, length vs. width, per-image box plots and CDFs. Downloads: CSV, Excel (sheets *Summary*, *Wires*, *Per image*, *Settings*), each figure as PNG (300 dpi), SVG or PDF, and a ZIP of everything, including overlays and the exact analysis settings for reproducibility.

Headless use, for example training on a GPU server, is possible with `python -m aunw.cli` and the sub-commands `generate`, `train`, `finetune`, `evaluate` and `analyze`. All processing is local: the server listens only on `localhost` and usage statistics are disabled.

Screenshots

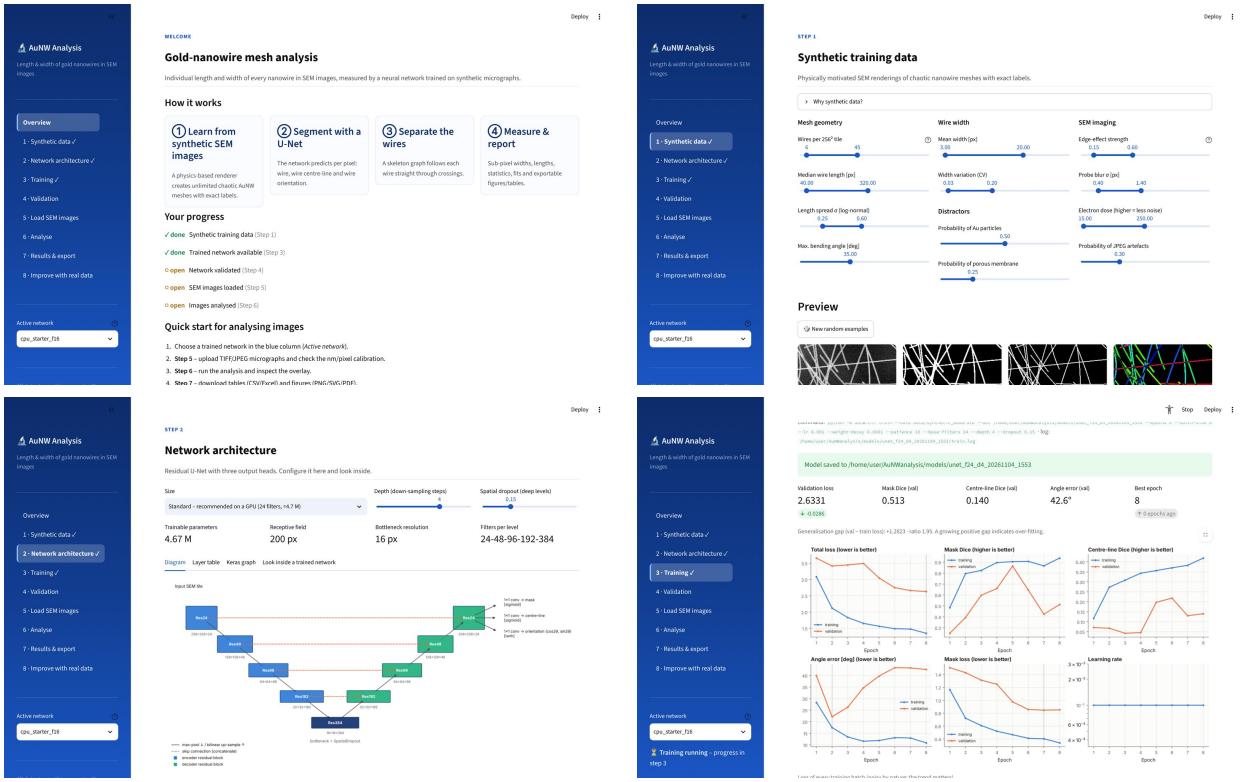


Figure 7: Overview, synthetic data generator (step 1), network architecture (step 2) and live training monitor (step 3, shown here for a deliberately tiny training set, so the over-fitting gap is visible).

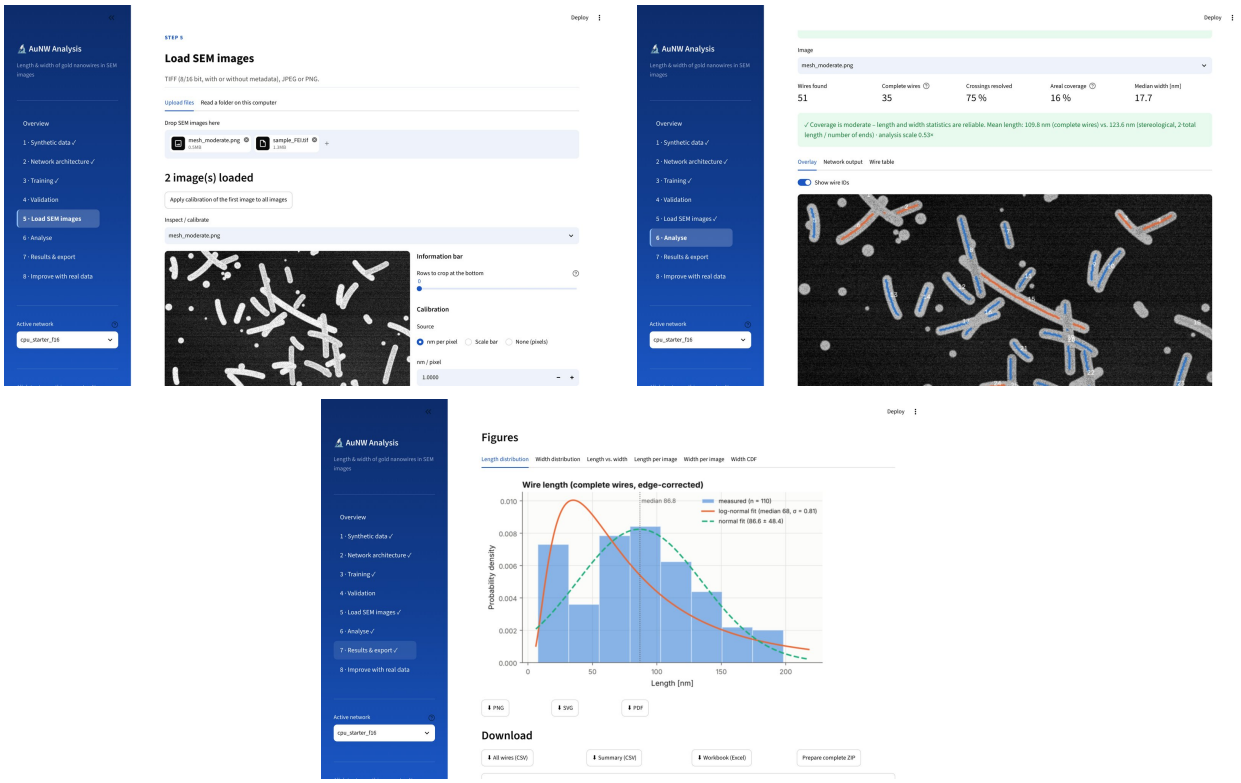


Figure 8: Loading and calibrating images (step 5), analysis with overlay, coverage-based reliability check and the two mean-length estimates (step 6), and results with fitted distributions and exports (step 7).

13 Proposed additional features (for discussion)

The current version focuses on length and width, as agreed. The following quantities can be derived from the existing graph and maps with little extra effort and are relevant for AuNW networks such as transparent electrodes:

1. **Orientation distribution and order parameter** $S = |\langle e^{2i\theta} \rangle|$ (0 isotropic, 1 aligned), relevant for anisotropic conductivity.
2. **Wire density** n (wires per µm^2) and **areal coverage**. Together with $\langle L \rangle$ they give the stick-percolation parameter nL^2, to be compared with the threshold $n_c L^2 \approx 5.64$ [7].
3. **Junction density and coordination number** (contacts per wire), which govern the junction-resistance-dominated sheet resistance.
4. **Pore-size distribution** of the mesh (sizes of background regions enclosed by wires), relevant for optical transmittance and haze.
5. **Curvature and tortuosity** $\tau = L/$end-to-end distance for flexible ultrathin wires.
6. **Width uniformity along a wire** (tapering, necking), from the individual profile widths w_i that are already computed.
7. **By-product analysis**: count and size distribution of Au particles, and the yield (wire area / total Au area).
8. **Bundles**: detection of parallel touching wires from width outliers and orientation.

References

[1] O. Ronneberger, P. Fischer, T. Brox. U-Net: Convolutional networks for biomedical image segmentation. *MICCAI* (2015).

[2] K. He, X. Zhang, S. Ren, J. Sun. Deep residual learning for image recognition. *CVPR* (2016).

[3] A. Odena, V. Dumoulin, C. Olah. Deconvolution and checkerboard artifacts. *Distill* (2016).

[4] I. Loshchilov, F. Hutter. Decoupled weight decay regularization. *ICLR* (2019).

[5] R. E. Miles. The sampling, by quadrats, of planar aggregates. *J. Microsc.* 113, 257 (1978).

[6] C. Lantuéjoul. On the estimation of mean values in individual analysis of particles. *Microsc. Acta* 5, 266 (1980).

[7] J. Li, S.-L. Zhang. Finite-size scaling in stick percolation. *Phys. Rev. E* 80, 040104 (2009).

[8] L. Reimer. *Scanning Electron Microscopy*, 2nd ed., Springer (1998).